

Министерство образования Республики Беларусь

Учреждение высшего образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

ОТЧЕТ
по лабораторной работе №5
на тему

Работа с базой данных

Студенты группы 150503

Шарай П. Ю.
Соломко В. Ю.

Проверила

Герман Ю. О.

Минск 2023

Цель: изучить технику работы с базой данных в Scala.

Краткие теоретические сведения:

Драйвер - это программное обеспечение, которое позволяет приложению взаимодействовать с определенным типом базы данных. Он предоставляет интерфейс для приложения, который понимает СУБД и позволяет выполнять операции с базой данных. Драйверы бывают разные для разных СУБД (например, драйвер для MySQL, PostgreSQL, SQLite и так далее).

SQL (Structured Query Language) - это язык, используемый для работы с реляционными базами данных. С его помощью можно выполнять различные операции, такие как создание и изменение таблиц, добавление и извлечение данных и т.д.

Выполнение работы:

Задание:

- 1.Расширить количество столбцов таблицы sclad, добавив поле количество товара на складе
- 2.Вывести товар, дающий максимальную прибыль (количество*цену).

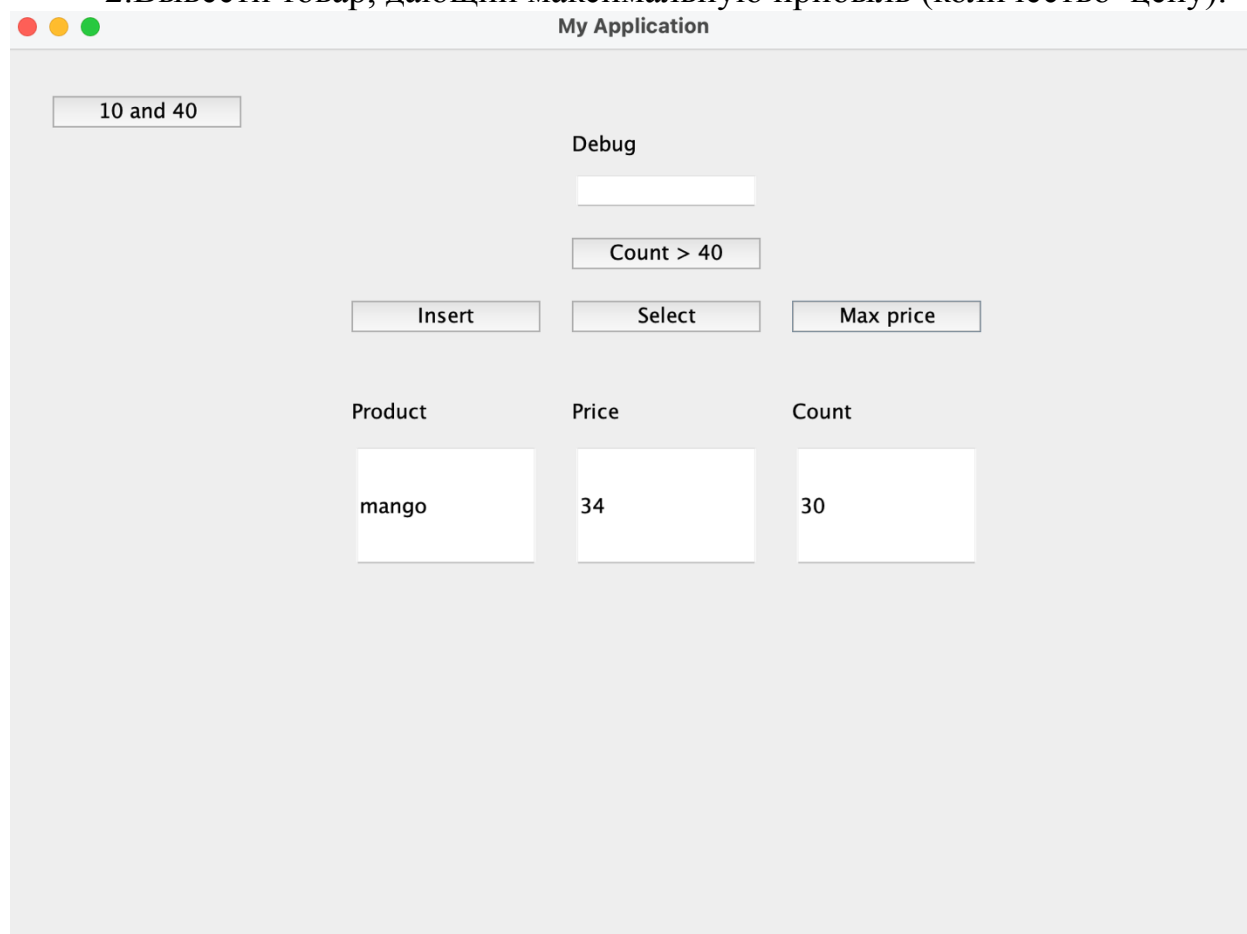


Рисунок 1– Результат выполнения 1 и 2 задания

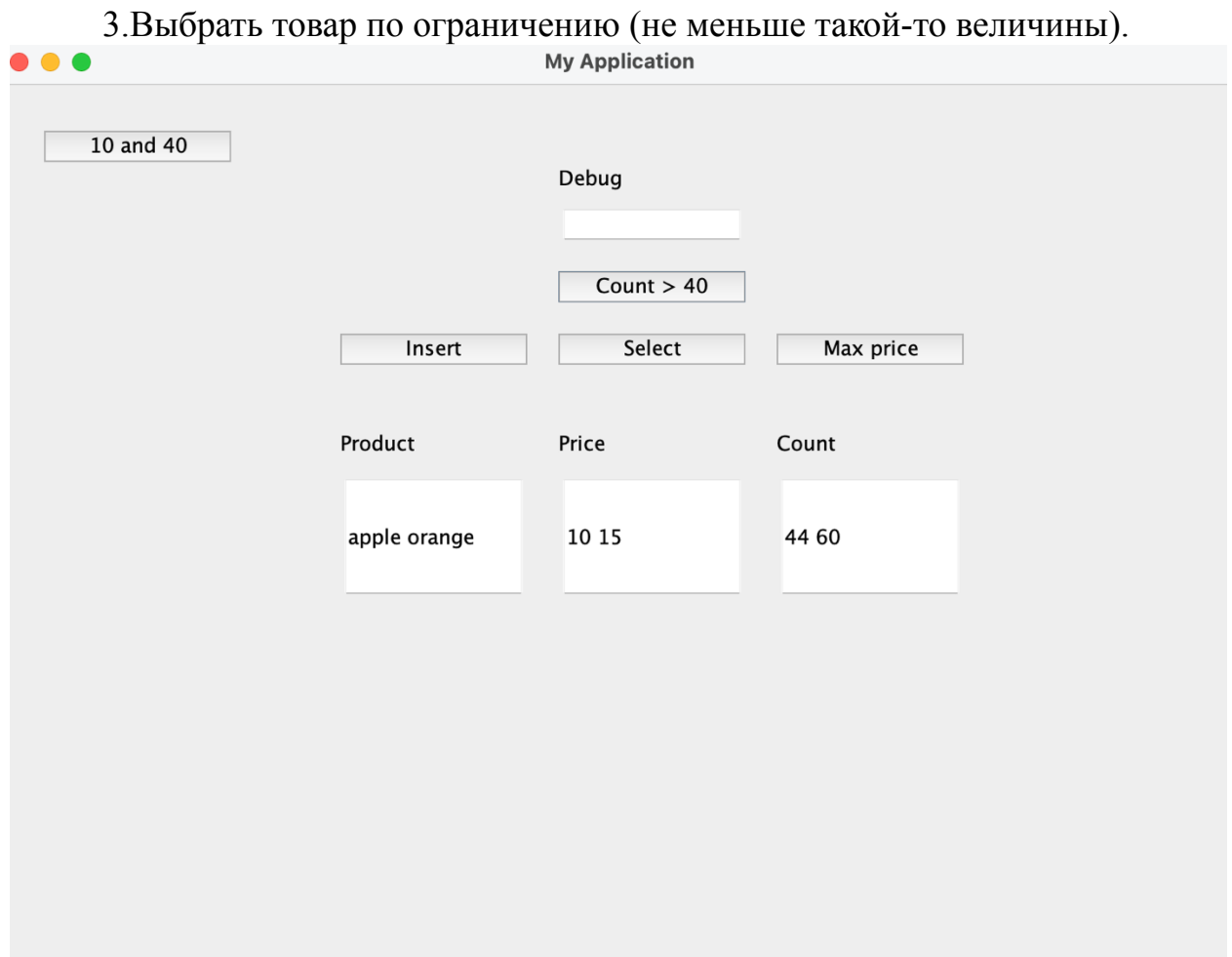


Рисунок 2 – Результат выполнения 3 задания

Листинг кода программы:

```
import java.awt.Color
import java.awt.event.{ActionEvent, ActionListener}
import java.sql.DriverManager
import javax.swing.*

object ButtonModule {
    val frame = new JFrame("My Application")
```

```

frame.setSize(680, 400)

frame.setLayout(null)

def main(args: Array[String]): Unit = {
    SwingUtilities.invokeLater(() => {
        val productLabel = new JLabel("Product")
        val priceLabel = new JLabel("Price")
        val countLabel = new JLabel("count_of_products")
        val moreThanLabel = new JLabel("More than (price)")
        productLabel.setBounds(20, 120, 130, 20)
        priceLabel.setBounds(160, 120, 130, 20)
        countLabel.setBounds(300, 120, 130, 20)
        moreThanLabel.setBounds(440, 120, 200, 20)

        val insertButton = new JButton("Insert")
        val selectButton = new JButton("Select")
        val getMaxButton = new JButton("Get max")
        val getMoreThanButton = new JButton("Get more than (price)")
        insertButton.setBounds(20, 80, 130, 20)
        selectButton.setBounds(160, 80, 130, 20)
        getMaxButton.setBounds(300, 80, 130, 20)
        getMoreThanButton.setBounds(440, 80, 200, 20)

        val productField = new JTextField(20)
        val priceField = new JTextField(20)
        val countField = new JTextField(20)
        val moreThanField = new JTextField(20)
        productField.setBounds(20, 150, 130, 20)
        priceField.setBounds(160, 150, 130, 20)
        countField.setBounds(300, 150, 130, 20)
        moreThanField.setBounds(440, 150, 200, 20)
    })
}

```

```

frame.add(insertButton)

frame.add(selectButton)

frame.add(getMaxButton)

frame.add(getMoreThanButton)


frame.add(productLabel)

frame.add(priceLabel)

frame.add(countLabel)

frame.add(moreThanLabel)


frame.add(productField)

frame.add(priceField)

frame.add(countField)

frame.add(moreThanField)


insertButton.addActionListener(new ActionListener{

    override def actionPerformed(e: ActionEvent): Unit = {

        val url = "jdbc:mysql://localhost:3306/mysql"

        val username = "root"

        val password = "petia2010"


        Class.forName("org.gjt.mm.mysql.Driver")


        val connection = DriverManager.getConnection(url, username,
password)


        try{

            val statement = connection.createStatement()

            val rs = statement.execute("INSERT INTO products VALUES
('" + productField.getText() + "','" + priceField.getText() + "','" +
countField.getText() + ")")

            productField.setText("")

```

```

        priceField.setText("")
        countField.setText("")
    }
    finally{
        connection.close()
    }
}
}))

selectButton.addActionListener(new ActionListener{
    override def actionPerformed(e: ActionEvent): Unit = {
        val url = "jdbc:mysql://localhost:3306/mysql"
        val username = "root"
        val password = "petia2010"

        Class.forName("org.gjt.mm.mysql.Driver")

        val connection = DriverManager.getConnection(url, username,
password)
        try{
            val statement = connection.createStatement()
            val rs = statement.executeQuery("SELECT * FROM products")

            println("Products:")
            while(rs.next()){
                val product = rs.getString("product")
                val price = rs.getInt("price")
                val count = rs.getInt("count_of_products")
                println(s"name=$product, price=$price,
count_of_products=$count")
            }
            println("")
        }
    }
})

```

```

        finally{
            connection.close()
        }
    }
})

val priceMoreThanOneButton = new JButton("Price More Than 1")
priceMoreThanOneButton.setBounds(160, 200, 180, 20)
frame.add(priceMoreThanOneButton)

priceMoreThanOneButton.addActionListener(new ActionListener {
    override def actionPerformed(e: ActionEvent): Unit = {
        val url = "jdbc:mysql://localhost:3306/mysql"
        val username = "root"
        val password = "petia2010"

        Class.forName("org.gjt.mm.mysql.Driver")

        val connection = DriverManager.getConnection(url,
username, password)

        try {
            val statement = connection.createStatement()
            val rs = statement.executeQuery("SELECT product,
price FROM products WHERE price > 1 GROUP BY product")

            println("Products with Price More Than 1:")
            while(rs.next()){
                val product = rs.getString("product")
                val price = rs.getDouble("price")
                println(s"Product=$product, Price=$price")
            }
            println("")
        }
    }
})

```

```

        finally {
            connection.close()
        }
    }
})

getMaxButton.addActionListener(new ActionListener{
    override def actionPerformed(e: ActionEvent): Unit = {
        val url = "jdbc:mysql://localhost:3306/mysql"
        val username = "root"
        val password = "petia2010"

        Class.forName("org.gjt.mm.mysql.Driver")

        val connection = DriverManager.getConnection(url, username,
password)

        try{
            val statement = connection.createStatement()
            val rs1 = statement.executeQuery("SELECT MAX(price *
count) AS m FROM products")

            var max: Int = 0
            while(rs1.next()){
                max = rs1.getInt("m")
            }

            val rs = statement.executeQuery("SELECT * FROM products
WHERE price * count_of_products = " + max)

            println("Max price * count_of_products:")
            while(rs.next()){
                val product = rs.getString("product")
                val price = rs.getInt("price")

```



```

        val count = rs.getInt("count_of_products")

        println(s"name=$product, price=$price,
count_of_products=$count")
    }

    println("")
}

finally{
    connection.close()
}

})

```

```

val avgPriceButton = new JButton("Average Price")
avgPriceButton.setBounds(20, 200, 130, 20)
priceMoreThanOneButton.setBounds(160, 200, 180, 20)

```

```

frame.add(avgPriceButton)
frame.add(priceMoreThanOneButton)

```

```

avgPriceButton.addActionListener(new ActionListener {
    override def actionPerformed(e: ActionEvent): Unit = {
        val url = "jdbc:mysql://localhost:3306/mysql"
        val username = "root"
        val password = "petia2010"

        Class.forName("org.gjt.mm.mysql.Driver")

        val connection = DriverManager.getConnection(url, username,
password)

        try {
            val statement = connection.createStatement()

```

```

        val rs = statement.executeQuery("SELECT AVG(price) AS
avg_price FROM products GROUP BY product")

        println("Average Price for Each Product:")
        while (rs.next()) {
            val product = rs.getString("product")
            val avgPrice = rs.getDouble("avg_price")
            println(s"Product=$product, Avg Price=$avgPrice")
        }
        println("")
    }
    finally {
        connection.close()
    }
}
}))

frame.setBackground(Color.BLUE)
frame.setLocationRelativeTo(null)
frame.setVisible(true)
}))
}
}

```

```

[mysql> select * from stock;
+-----+-----+-----+
| product | price | count_of_products |
+-----+-----+-----+
| apple   | 10    | 44                 |
| banana  | 100   | 7                  |
| mango   | 34    | 30                 |
| orange  | 15    | 60                 |
+-----+-----+-----+
4 rows in set (0.00 sec)

```

Рисунок 4 – Обзор базы данных

Вывод: В ходе выполнения данной лабораторной работы нами были проанализированы и изучены техники работы с базой данной в Scala.